Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Middleware

L'histoire d'une application qui grandit

Speaker notes

Fil rouge du cours : on part d'une application toute simple, et on ajoute un middleware chaque fois qu'un problème concret apparaît. Chaque mécanisme est déroulé étape par étape : prendre le temps sur chaque étape plutôt que de tout voir.

C'est quoi un middleware ?

- Un logiciel qui se place **entre** notre application et le reste du monde
- Ce n'est pas notre code, mais notre application ne marche pas sans
- Exemples : Nginx, PostgreSQL, RabbitMQ

Speaker notes

Demander d'abord à la salle s'ils en connaissent un. Une base de données est un middleware : on ne l'écrit pas, mais l'application ne marche pas sans.

FraisDirect

Une épicerie en ligne qui livre des produits frais



Un serveur, une application, une base. Tout va bien… pour l'instant.

Speaker notes

L'application fictive de tout le cours. Au départ, tout tient sur un seul serveur. On va la faire grandir en quatre étapes.

Au programme

1. Le site est saturé : **load balancer**
2. Plusieurs applications, une seule adresse : **reverse proxy**
3. Des données de toutes les formes : **bases de données**
4. Des services qui doivent se parler : **messaging**

Speaker notes

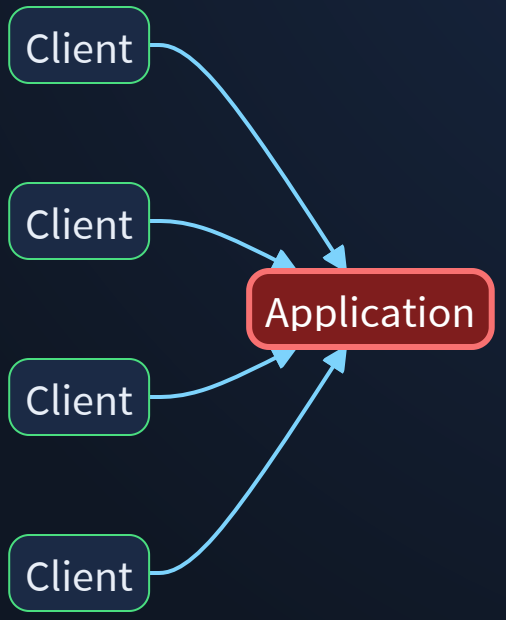
Chaque acte suit le même plan : un problème, puis le middleware qui le résout, puis comment il fonctionne étape par étape.

ACTE 1

Le Black Friday

Le site est saturé

Le problème

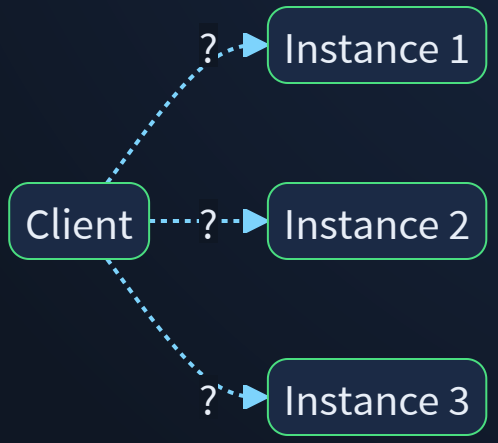


- 10 fois plus de clients que d'habitude
- Un seul serveur : il ne répond plus

Speaker notes

Demander aux étudiants ce qu'ils feraient. Réponses attendues : un plus gros serveur (scalabilité verticale, mais il y a une limite et ça reste un seul serveur), ou plusieurs serveurs (scalabilité horizontale).

Étape 1 : lancer plusieurs instances

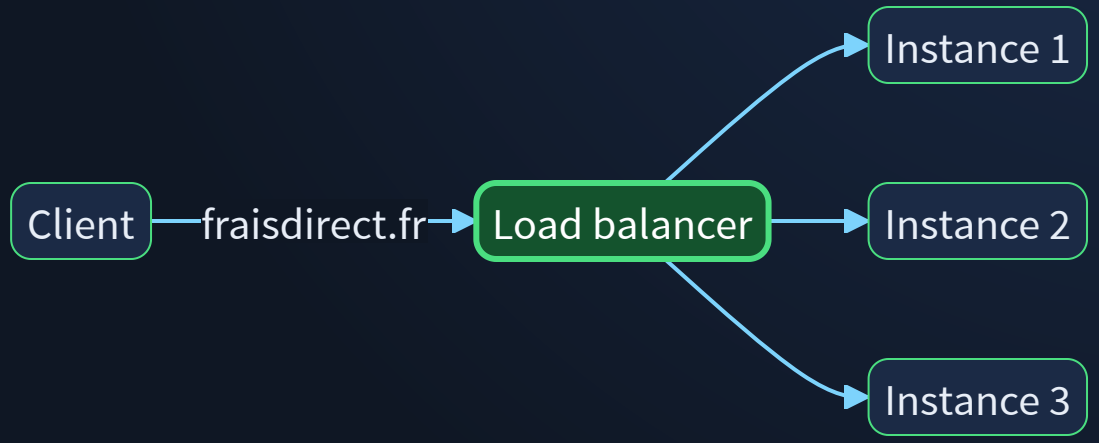


Mais le client ne connaît qu'une adresse : `fraisdirect.fr`.
Sur quelle instance doit-il aller ?

Speaker notes

Les trois instances font tourner exactement le même code. Le problème : le client ne doit pas avoir à choisir.

Étape 2 : un load balancer devant



L'adresse `fraisdirect.fr` pointe sur le load balancer. Lui seul connaît les instances.

Speaker notes

Le load balancer reçoit toutes les requêtes et les transmet aux instances. Pour le client, rien ne change.

Étape 3 : répartir les requêtes

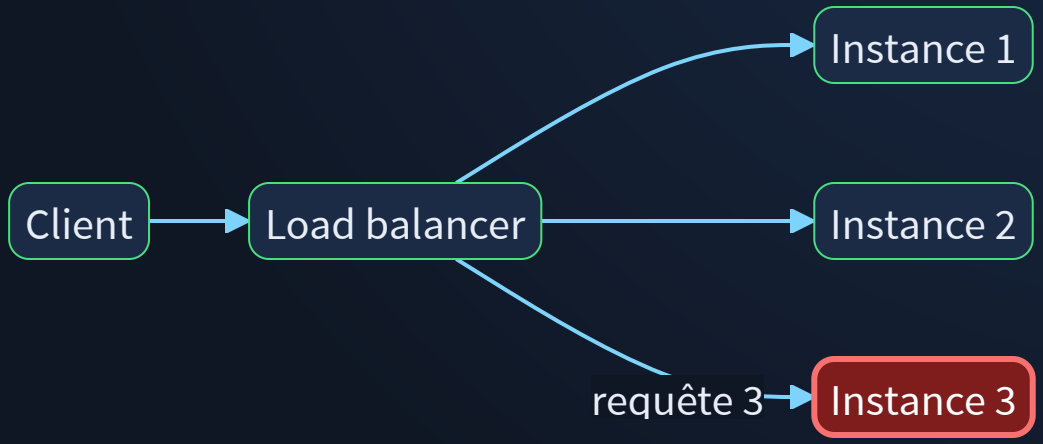
Le plus simple : chacun son tour (*round robin*)

- Requête 1 → instance 1
- Requête 2 → instance 2
- Requête 3 → instance 3
- Requête 4 → instance 1 à nouveau

Speaker notes

Il existe d'autres façons de répartir (vers l'instance la moins chargée, toujours la même instance pour un client), mais round robin suffit pour comprendre.

Et si une instance tombe ?

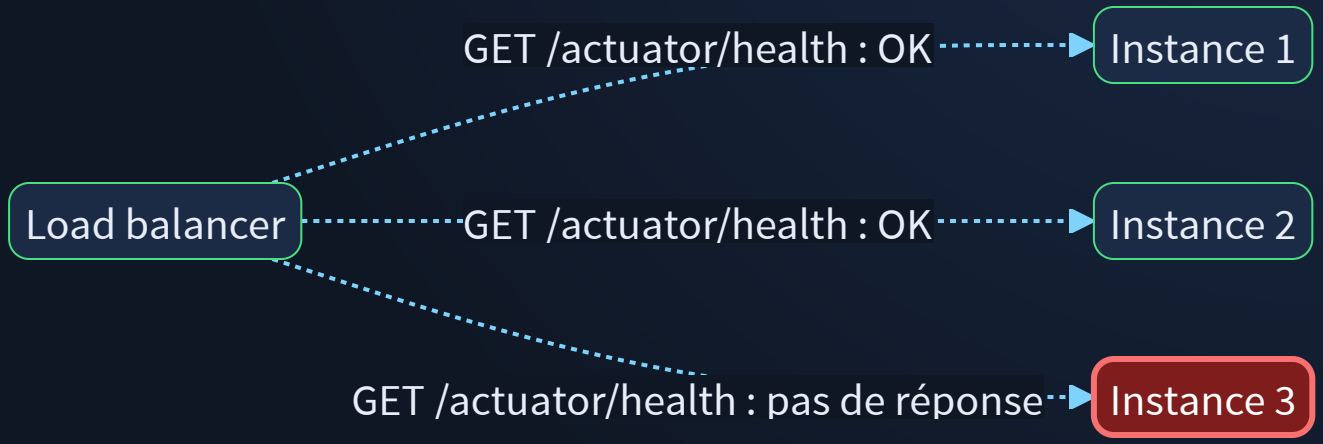


Une requête sur trois part vers une instance morte : le client a une erreur.

Speaker notes

Il faut que le load balancer sache quelles instances sont en vie. C'est l'objet des slides suivantes.

Étape 4 : le health check

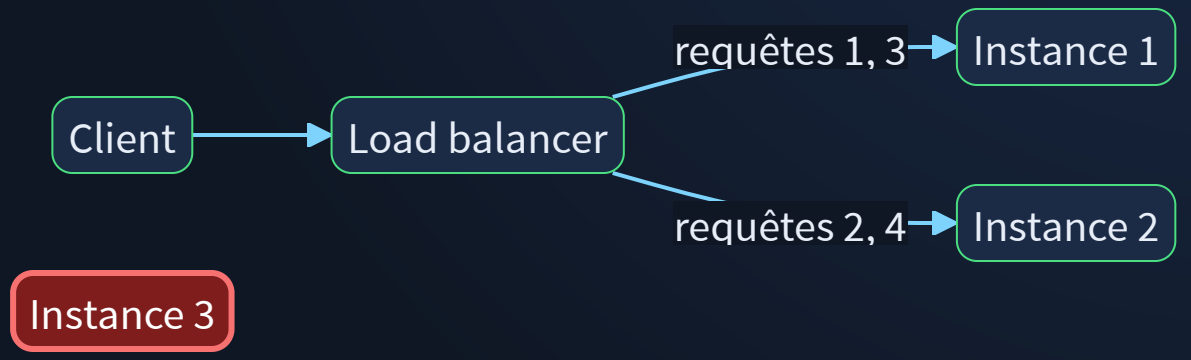


Toutes les quelques secondes, le load balancer interroge chaque instance.

Speaker notes

Avec Spring Boot, l'URL /actuator/health est fournie par spring-boot-starter-actuator. En général, on retire une instance après plusieurs échecs, pas dès le premier.

Étape 5 : retirer l'instance



Plus aucune requête ne part vers l'instance 3.
Quand elle répond de nouveau, le load balancer la réintègre.

Speaker notes

C'est aussi ce qui permet de mettre à jour l'application sans coupure : on arrête et on met à jour les instances une par une.

QUESTION

Le panier est gardé en mémoire dans la session HTTP de l'application. Avec 3 instances derrière un load balancer en round robin, ça marche ?

Speaker notes

Réponse : non. Laisser répondre, puis dérouler la slide suivante étape par étape.

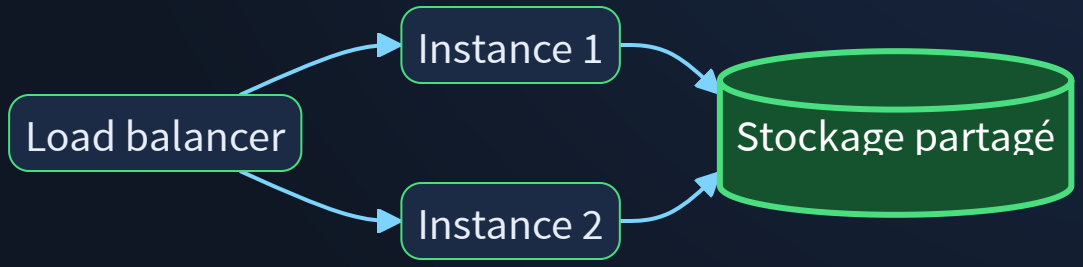
Le panier perdu, étape par étape

1. « Ajouter des tomates » → instance 1
   le panier est rangé dans la mémoire de l'instance 1
2. « Voir mon panier » → instance 2
3. L'instance 2 ne connaît pas ce panier : il est vide

Speaker notes

Chaque instance a sa propre mémoire. Rien n'est partagé entre elles.

Étape 6 : l'état hors de l'application



Les instances ne gardent rien en mémoire : elles sont **stateless**.

Quel stockage ? Réponse à l'acte 3.

Speaker notes

N'importe quelle instance peut traiter n'importe quelle requête. On y revient avec les bases clé-valeur (Redis).

À retenir

- Le load balancer est la seule adresse connue des clients
- Il répartit les requêtes entre les instances
- Il retire les instances qui ne répondent plus
- Les instances doivent être stateless

Exemples : HAProxy, Nginx, les load balancers des clouds

Speaker notes

En pratique, les étudiants utiliseront surtout le load balancer de leur cloud ou le Service Kubernetes, sans l'installer eux-mêmes.

ACTE 2

Un site, plusieurs applications

Une seule adresse pour tout le monde

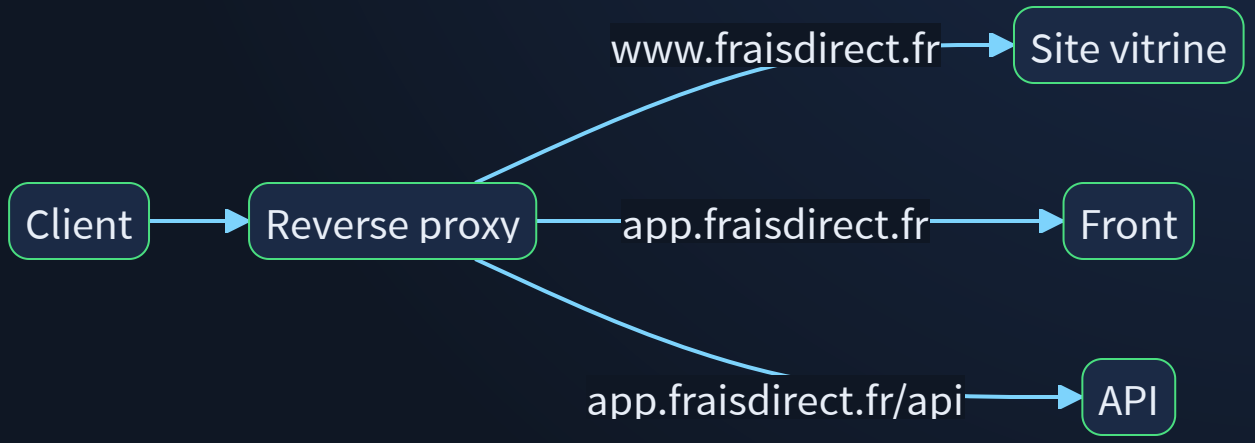
Le problème

- FraisDirect a maintenant trois applications : un site vitrine, un front et une API
- Les clients ne doivent voir qu'une seule adresse

Speaker notes

On ne veut pas exposer trois serveurs ou trois ports aux clients.

Le reverse proxy



Il reçoit toutes les requêtes et choisit l'application selon le domaine et le chemin.

Speaker notes

Le client ne sait pas qu'il y a trois applications derrière.

Suivre une requête

`GET https://app.fraisdirect.fr/api/commandes`

1. Le reverse proxy reçoit la requête
2. Il lit le domaine : `app.fraisdirect.fr`
3. Il lit le chemin : commence par `/api`
4. Il transmet la requête à l'API
5. Il renvoie la réponse de l'API au client

Speaker notes

Faire dérouler la même chose aux étudiants avec https://app.fraisdirect.fr/panier : même domaine, mais le chemin ne commence pas par /api, donc le front.

La même chose avec Nginx

```
server {
    server_name app.fraisdirect.fr;

    location /api/ {
        proxy_pass http://api:8080;
    }

    location / {
        proxy_pass http://front:80;
    }
}
```

Speaker notes

Lire la configuration dans l'ordre des étapes de la slide précédente : le domaine (server_name), puis le chemin (location), puis la destination (proxy_pass). Nginx choisit la location la plus précise : /api/ avant /.

Bonus : HTTPS à un seul endroit

- Le client parle en HTTPS au reverse proxy
- Le reverse proxy déchiffre la requête
- Il la transmet en HTTP aux applications, sur le réseau interne

Un seul certificat à gérer, au lieu d'un par application.

Speaker notes

Le reverse proxy peut aussi gérer l'authentification, le cache, la compression, la limitation du nombre de requêtes. Traefik et Caddy obtiennent les certificats Let's Encrypt automatiquement.

Load balancer ou reverse proxy ?

- Souvent le même logiciel : Nginx fait les deux
- Le reverse proxy découpe le site en applications : préoccupation des DEV
- Le load balancer tient la charge et les pannes : préoccupation des OPS

Exemples : Nginx, Traefik, Caddy, HAProxy

Speaker notes

La différence est dans l'intention, pas dans l'outil.

QUESTION

Ajouter une 2e instance de l'application pour encaisser le pic du Black Friday : c'est d'abord le rôle…

A. du reverse proxy (préoccupation DEV)

B. du load balancer (préoccupation OPS)

Speaker notes

Réponse : B. Plusieurs instances d'une même application pour tenir la charge, c'est l'acte 1.

ACTE 3

Des données de toutes les formes

La bonne base pour le bon besoin

Les besoins de FraisDirect

- Les commandes
- Le catalogue de produits
- Les paniers (promis à l'acte 1)
- La température des chambres froides
- Les photos des produits

Speaker notes

Faire deviner, pour chaque besoin, si PostgreSQL convient.

La méthode

Pour chaque besoin, trois questions :

1. À quoi ressemblent les données ?
2. Comment les lit-on ?
3. Est-ce grave d'en perdre ?

Speaker notes

On applique ces trois questions à chaque besoin, un par un. C'est cette méthode qu'il faut retenir, plus que la liste des bases.

Les commandes

commande	client	date
1042	Alice	24/11

commande	produit	quantité
1042	Tomates	2 kg
1042	Comté	300 g

1. Des lignes et des colonnes, liées entre elles
2. On les croise : les commandes d'un client, avec leurs lignes
3. Très grave : c'est de l'argent

Speaker notes

Faire répondre aux trois questions avant de montrer les fragments.

Commandes : base relationnelle

- Des tables liées entre elles, interrogées en SQL
- ⊕ Transactions : tout est enregistré, ou rien
- ⊕ La base refuse les données incohérentes
- ⊖ Changer la structure demande une migration

PostgreSQL, MySQL / MariaDB, Oracle

Speaker notes

Transaction : si on enregistre la commande mais que l'enregistrement des lignes échoue, rien n'est gardé. C'est le choix par défaut pour les données métier.

Le catalogue

```
{ "nom": "Comté 18 mois", "prix": 32.5, "affinage": "18 mois", "lait
{ "nom": "Tomate cœur de bœuf", "prix": 4.2, "origine": "Provence",
```

1. Chaque produit a ses propres champs
2. On cherche par mots : « tomate provence », même avec une faute
3. Peu grave : on peut le reconstruire

Speaker notes

En relationnel, un fromage et une tomate n'ont pas les mêmes colonnes : il faudrait une colonne par attribut possible, ou une table générique pénible à interroger.

Catalogue : base document

- Chaque donnée est un document JSON
- ⊕ Chaque document a ses propres champs
- ⊕ Recherche avancée (fautes de frappe, filtres)
- ⊖ Pas de jointures : on duplique les données

MongoDB, Elasticsearch

Speaker notes

Elasticsearch est d'abord un moteur de recherche : on l'alimente à partir de la base principale.

Les paniers

```
panier:alice → { "tomates": "2 kg", "comté": "300 g" }
panier:bob   → { "pain": 1 }
```

1. Une clé, une valeur
2. Toujours lu par sa clé, très souvent, très vite
3. Pas très grave : le client refait son panier

Speaker notes

On ne cherche jamais « tous les paniers qui contiennent des tomates » : on lit toujours le panier d'un client donné.

Paniers : base clé-valeur

- Comme une `Map`, partagée entre les instances
- ⊕ Très rapide : les données sont en mémoire
- ⊕ Une clé peut expirer toute seule (panier abandonné)
- ⊖ On ne lit que par la clé : pas de recherche

Redis / Valkey, Memcached

Speaker notes

C'est le stockage partagé de l'acte 1 : les instances deviennent stateless. Avec Spring, Spring Session range les sessions HTTP dans Redis sans changer le code. Redis peut aussi écrire ses données sur disque, si on le configure.

QUESTION

La température d'un capteur, relevée toutes les 10 secondes pendant un an : quel type de BDD ?

A. Relationnel

B. Document

C. Timeseries

D. Clé-valeur

Speaker notes

Réponse : C. Les étudiants ne connaissent pas encore ce type : la question sert à les faire réfléchir. Les slides suivantes expliquent.

Les chambres froides

heure	capteur	température
10:00:00	frigo-1	3,1 °C
10:00:10	frigo-1	3,2 °C
10:00:20	frigo-1	3,2 °C

1. Une mesure datée, encore et encore : 3 millions par an et par capteur
2. Par période : « la moyenne par heure cette semaine »
3. Une mesure perdue n'est pas grave

Speaker notes

Usage typique : une alerte si la température dépasse 4 °C pendant plus de 10 minutes, et un graphique de la semaine.

Chambres froides : base timeseries

- Faite pour des mesures rangées dans le temps
- ⊕ Compresse énormément les mesures
- ⊕ Calcule vite des moyennes par période
- ⊖ Ne sait faire que ça

InfluxDB, TimescaleDB, Prometheus

Speaker notes

TimescaleDB est une extension de PostgreSQL : pratique si PostgreSQL est déjà en place. Prometheus sert surtout à surveiller les applications.

Les photos des produits

```
fraisdirect-photos/tomate-coeur-de-boeuf.jpg
fraisdirect-photos/comte-18-mois.jpg
```

1. Des fichiers, parfois gros
2. On les télécharge en entier, par leur nom
3. Gênant : il faut les sauvegarder

Speaker notes

Pourquoi pas le disque du serveur ? Avec trois instances, chacune a son propre disque : une photo envoyée sur l'instance 1 n'existe pas sur l'instance 2. Même problème que le panier de l'acte 1.

Photos : object storage

- Des fichiers (*objets*) rangés dans des *buckets*
- ⊕ Accessible en HTTP depuis toutes les instances
- ⊕ Pratiquement sans limite de taille
- ⊖ On remplace un fichier, on ne le modifie pas

AWS S3, Google Cloud Storage, Garage, Ceph

Speaker notes

Le bucket est le conteneur : fraisdirect-photos dans l'exemple. L'API de S3 est devenue un standard, que les autres outils imitent.

Récapitulatif

Besoin	Type	Exemple
Commandes	Relationnel	PostgreSQL
Catalogue	Document	MongoDB, Elasticsearch
Paniers	Clé-valeur	Redis
Températures	Timeseries	InfluxDB
Photos	Object storage	S3

Speaker notes

Il existe d'autres familles, comme les bases colonne (Cassandra, ClickHouse) pour les très gros volumes. Hors programme ici.

QUESTION

Une application avec peu de trafic, beaucoup de règles métier et une recherche par nom de produit : quel type de SGBD choisir ?

A. Relationnel

B. Document

C. Clé-valeur

D. Timeseries

Speaker notes

Réponse : A. Une recherche par nom se fait très bien en SQL. Ceux qui répondent B pensent à la recherche : c'est la transition vers la slide suivante.

Une base de plus, c'est un coût

- Il faut l'installer, la sauvegarder, la surveiller, la mettre à jour
- PostgreSQL sait déjà chercher du texte et stocker du JSON
- On commence avec une base relationnelle, on en ajoute une autre quand le besoin est réel

Speaker notes

Le piège classique : une base par besoin dès le premier jour.

Bonnes pratiques

- Une base ne se partage pas entre applications différentes
- Une base se sauvegarde… et on teste la restauration
- La bonne base pour le bon usage, mais pas une de plus que nécessaire

Speaker notes

Base partagée entre applications : le schéma devient un contrat que personne ne peut plus modifier. Une sauvegarde dont on n'a jamais testé la restauration ne compte pas.

ACTE 4

Des services qui doivent se parler

Le messaging

Le problème

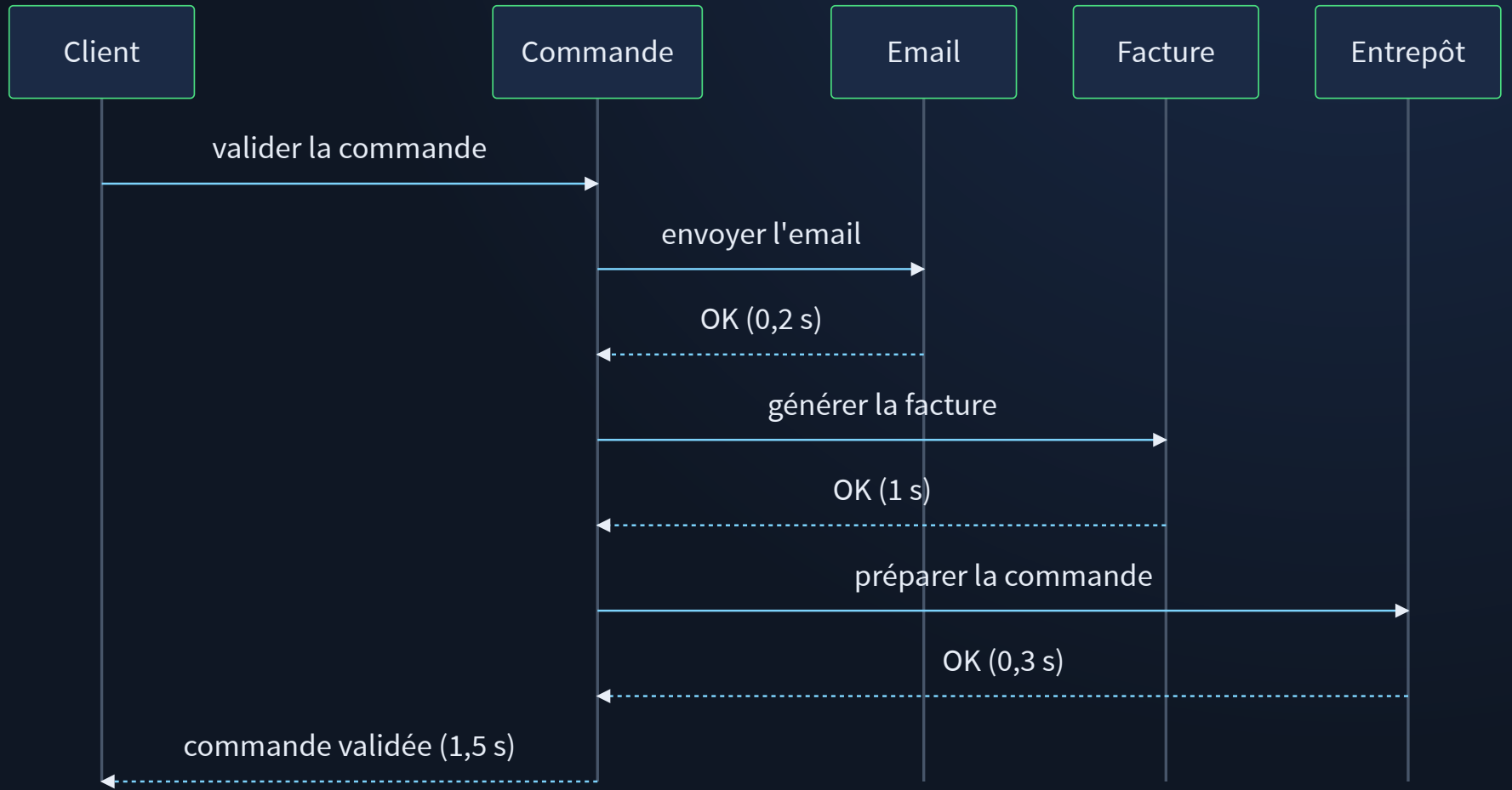
Quand une commande est validée, il faut aussi :

- envoyer un email de confirmation
- générer la facture
- prévenir l'entrepôt

Speaker notes

Ces trois tâches sont faites par trois autres services. Comment le service commande les prévient-il ?

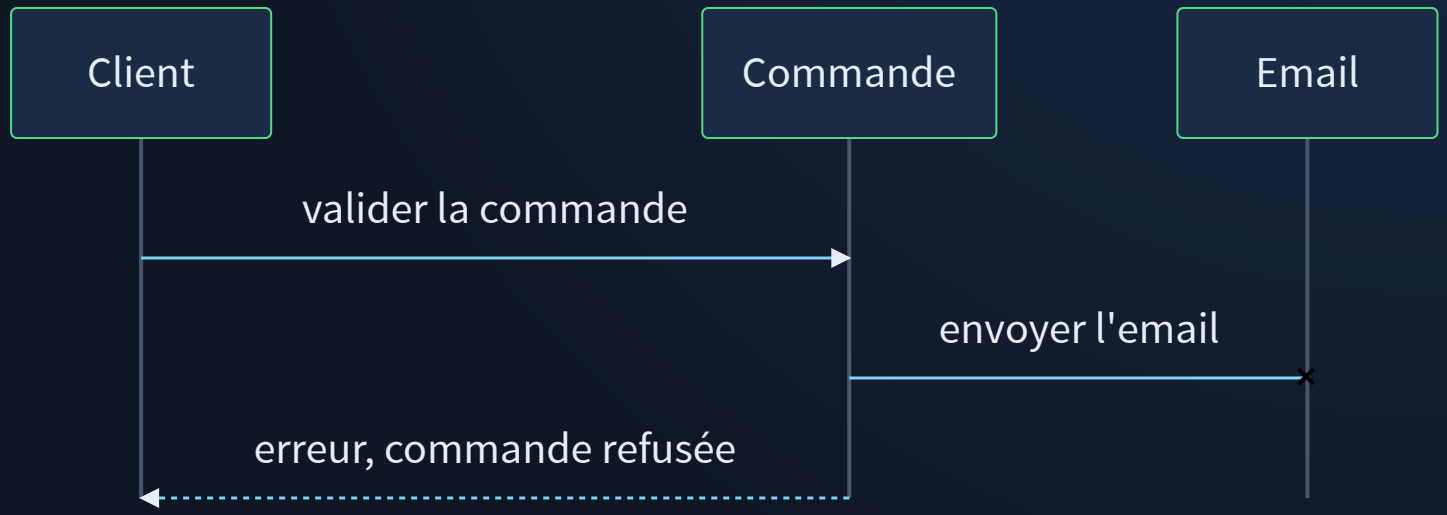
Première idée : des appels HTTP



Speaker notes

Dérouler le diagramme de haut en bas. Le client attend que les trois services aient répondu, l'un après l'autre.

Et si l'email est en panne ?

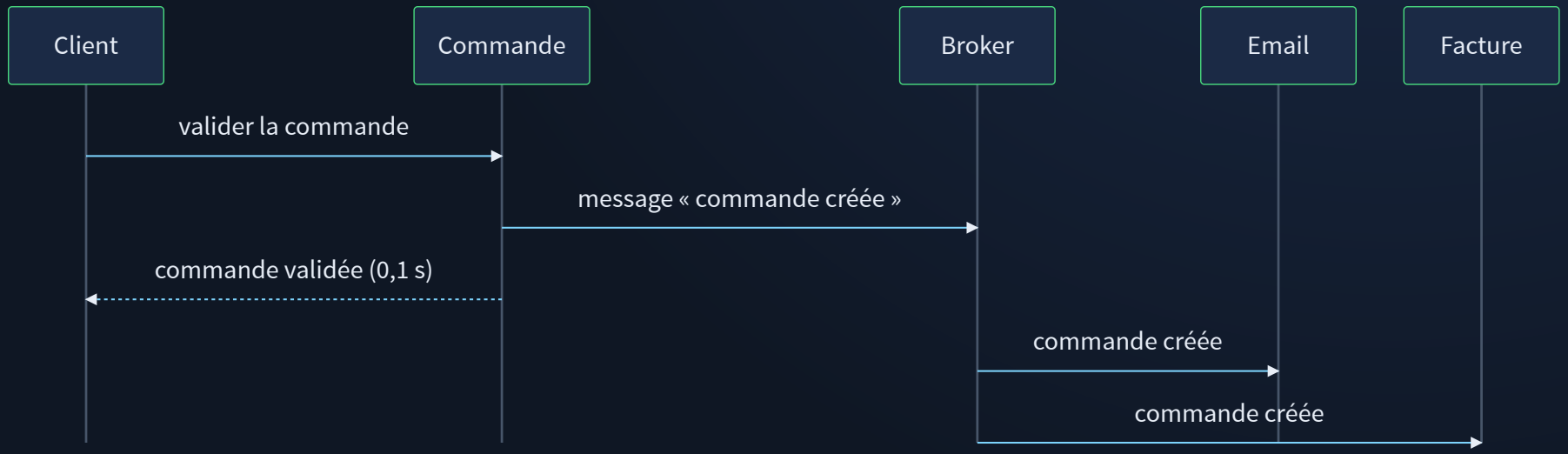


Une panne de l'email empêche de vendre !

Speaker notes

Deux problèmes : le client attend tout le monde, et la panne d'un service secondaire bloque la vente.

Deuxième idée : un broker de messages



Speaker notes

Le service commande dépose un message et répond tout de suite au client. Le broker le distribue ensuite aux services intéressés, à leur rythme.

Ce qu'on gagne

- **Rapide** : le client n'attend plus les autres services
- **Robuste** : si l'email est en panne, son message l'attend dans le broker
- **Évolutif** : un nouveau service s'abonne sans toucher au service commande

Speaker notes

Le service commande annonce un fait (« une commande a été créée ») ; il ne donne pas d'ordre et ne sait pas qui écoute.

Bonus : absorber les pics



Pendant le pic, la file se remplit. Après, le service facture rattrape son retard. Rien n'est perdu.

Speaker notes

Si le consumer reste durablement plus lent que le producer, la file grossit sans fin : il faut alors ajouter des consumers.

Le vocabulaire



- **Producer** : envoie le message
- **Exchange** : le tri postal, il distribue les messages
- **Queue** : la boîte aux lettres, elle garde les messages
- **Consumer** : lit les messages d'une queue

Speaker notes

Vocabulaire AMQP, le protocole de RabbitMQ. Le binding est la règle qui relie une queue à un exchange. La routing key est l'adresse écrite sur le message.

Le trajet d'un message (1/5)



Le producer envoie le message à l'**exchange**, avec la routing key `commande.creee`.

Speaker notes

Le producer ne connaît ni les queues ni les consumers : seulement l'exchange et la routing key.

Le trajet d'un message (2/5)



L'exchange compare la routing key aux **bindings** de chaque queue.

Speaker notes

Les deux queues sont intéressées par les commandes créées : leurs bindings correspondent.

Le trajet d'un message (3/5)



Il dépose **une copie** du message dans chaque queue qui correspond.

Speaker notes

Les messages attendent dans les queues, même si les consumers sont arrêtés.

Le trajet d'un message (4/5)



Chaque consumer lit le message dans sa queue et le traite.

Speaker notes

Chaque service avance à son rythme : la facture peut être générée avant ou après l'email.

Le trajet d'un message (5/5)



Le traitement a réussi : le consumer confirme (**ack**), la queue supprime le message.

Speaker notes

Tant que le message n'est pas confirmé, la queue le garde. On verra ce qui se passe quand le traitement échoue.

La routing key

Des mots séparés par des points :

`commande.creee.fr`

Mot 1	Mot 2	Mot 3
commande	creee	fr
quoi	quel événement	quel pays

Speaker notes

Le sens des mots est une convention choisie par l'équipe. Le broker, lui, ne voit que des mots.

Le binding : mot à mot

Binding `commande.creee.fr`, message `commande.creee.fr`

Binding	Message	
commande	commande	✓
creee	creee	✓
fr	fr	✓

Tous les mots correspondent : la queue reçoit le message.

Speaker notes

Le cas le plus simple : le binding est identique à la routing key. On ajoute ensuite les jokers.

Le joker `*` : exactement un mot

Binding `commande.*.fr`, message `commande.creee.fr`

Binding	Message	
commande	commande	✓
*	creee	✓ un mot
fr	fr	✓

Cette queue reçoit tous les événements des commandes françaises.

Speaker notes

commande.annulee.fr arrive aussi dans cette queue. commande.creee.be n'y arrive pas.

Le joker `#` : zéro, un ou plusieurs mots

Binding `commande.#`, message `commande.creee.fr`

Binding	Message	
commande	commande	✓
#	creee.fr	✓ deux mots

Cette queue reçoit tout ce qui commence par `commande`.

Speaker notes

Ce ne sont pas des expressions régulières : * et # remplacent des mots entiers, jamais un morceau de mot.

QUESTION

Une queue est liée avec le binding `commande.*`. Quels messages reçoit-elle ?

A. `commande.creee` seulement

B. `commande.creee.fr` seulement

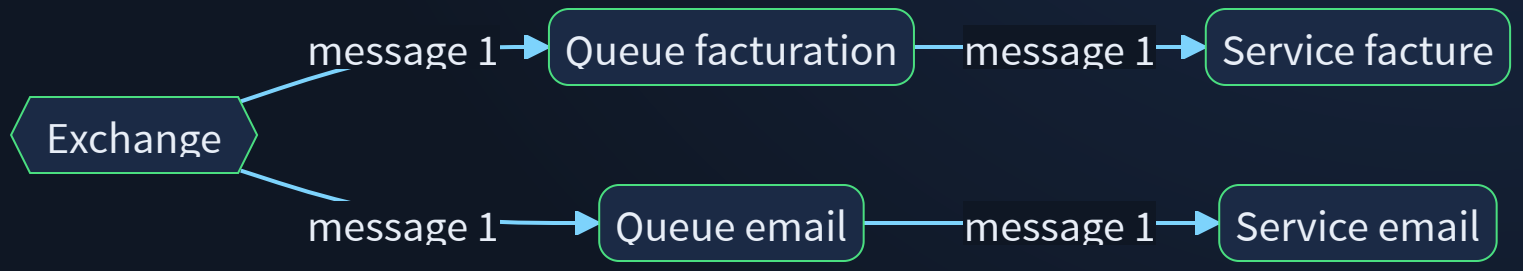
C. Les deux

D. Aucun des deux

Speaker notes

Réponse : A. Refaire la comparaison mot à mot au tableau : pour commande.creee.fr, il reste le mot fr sans rien en face dans le binding. Pour recevoir les deux, il faudrait commande.#.

Plusieurs queues : chacun sa copie

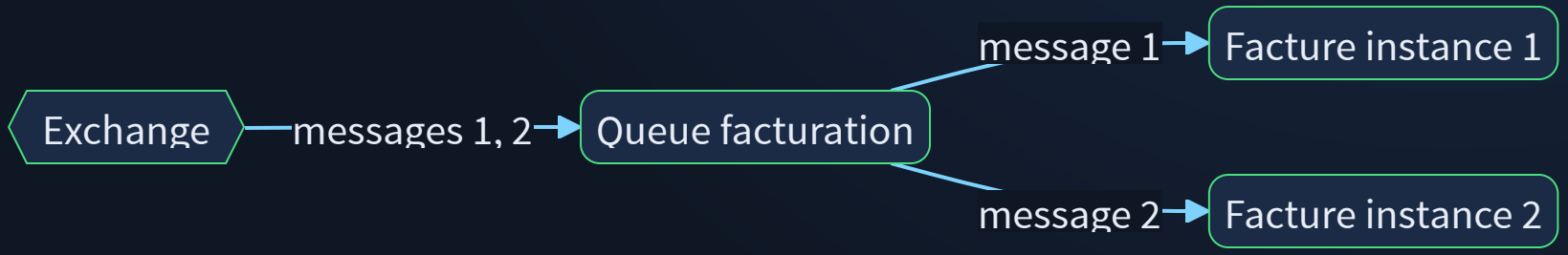


Deux services différents : chacun reçoit le message.

Speaker notes

C'est le cas vu dans le trajet d'un message.

Une queue, plusieurs instances : on se partage



Deux instances du même service : chaque message est traité **une seule fois**.

Speaker notes

Lien avec l'acte 1 : on peut lancer plusieurs instances du service facture pour aller plus vite, sans générer deux factures pour la même commande.

Des brokers

- RabbitMQ : celui de ce cours
- Mosquitto : pour les objets connectés (les capteurs des chambres froides)
- Kafka : pour de très gros volumes

Speaker notes

Kafka fonctionne différemment : les messages ne sont pas supprimés après lecture et peuvent être relus. Hors programme ici.

ACTE 4 · PRATIQUE

RabbitMQ dans Spring

En 7 étapes

Ce qu'on va construire



1. La dépendance
2. Les messages en JSON
3. L'exchange
4. La queue
5. Le binding
6. Envoyer un message
7. Recevoir un message

Speaker notes

Garder ce schéma en tête : chaque étape construit un des éléments.

Étape 1 : la dépendance

```
implementation 'org.springframework.boot:spring-boot-starter-amqp'
```

Et l'adresse du broker :

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
```

Speaker notes

Pour lancer RabbitMQ en local : docker run -p 5672:5672 -p 15672:15672 rabbitmq:management. L'interface web sur le port 15672 (guest / guest) permet de voir les exchanges, les queues et les messages : très utile pour suivre les étapes suivantes.

Étape 2 : les messages en JSON

```
@Bean
MessageConverter jsonMessageConverter() {
    return new Jackson2JsonMessageConverter();
}
```

Spring transforme nos objets Java en JSON à l'envoi, et le JSON en objets à la réception.

Speaker notes

Sans ce bean, on ne peut envoyer que des String ou des objets sérialisés en binaire Java. Spring Boot branche ce bean automatiquement. Avec Spring Boot 4 (Jackson 3), la classe s'appelle JacksonJsonMessageConverter.

Étape 3 : l'exchange

```
@Bean
TopicExchange fraisDirectExchange() {
    return new TopicExchange("fraisdirect");
}
```

Un exchange de type *topic* : il comprend les jokers `*` et `#`.

Speaker notes

Spring crée l'exchange dans RabbitMQ au démarrage s'il n'existe pas. Le montrer dans l'interface web.

Étape 4 : la queue

```java
public static final String FACTURATION_QUEUE = "facturation";

@Bean
Queue facturationQueue() {
    return QueueBuilder.durable(FACTURATION_QUEUE).build();
}
```

durable : la queue survit au redémarrage de RabbitMQ.

Speaker notes

La constante sera réutilisée par le listener. Pour aller plus loin : les quorum queues (.quorum()) sont répliquées sur plusieurs serveurs RabbitMQ.

Étape 5 : le binding

```
@Bean
Binding facturationBinding(
        Queue facturationQueue,
        TopicExchange fraisDirectExchange) {
    return BindingBuilder.bind(facturationQueue)
            .to(fraisDirectExchange)
            .with("commande.creee.*");
}
```

La queue `facturation` reçoit toutes les commandes créées, quel que soit le pays.

Speaker notes

Se lit comme une phrase : relier la queue facturation à l'exchange fraisdirect, avec le motif commande.creee.*. Les étapes 2 à 5 vont dans une même classe @Configuration.

Étape 6 : envoyer un message

```java
@Service
public class CommandeService {

    private final RabbitTemplate rabbitTemplate;
    // ...

    public void valider(Commande commande) {
        commandeRepository.save(commande);
        rabbitTemplate.convertAndSend("fraisdirect",
                "commande.creee." + commande.pays(),
                new CommandeCreee(commande.id()));
    }
}
```

Speaker notes

convertAndSend prend trois choses : l'exchange, la routing key et l'objet. On envoie un petit objet avec les informations utiles, pas l'entité JPA entière.

Étape 7 : recevoir un message

```java
@Service
public class FacturationListener {

    private final FactureService factureService;
    // ...

    @RabbitListener(queues = FACTURATION_QUEUE)
    public void commandeCreee(CommandeCreee commande) {
        factureService.generer(commande);
    }
}
```

Speaker notes

Ligne 7 : on écoute une queue, jamais un exchange. Ligne 8 : le JSON est transformé en CommandeCreee. Ligne 9 : le traitement. Si la méthode se termine normalement, Spring envoie l'ack tout seul.

QUESTION

À vous : consommez la file `commandes`

Chaque message reçu (une chaîne) doit être passé à `commandeService.traiter(...)`.

Speaker notes

Réponse attendue : @RabbitListener(queues = "commandes") sur une méthode public void recevoir(String commande) { commandeService.traiter(commande); }. Erreurs fréquentes : oublier l'annotation, mettre le nom de l'exchange au lieu de celui de la queue.

ACTE 4 · ERREURS

Quand le traitement échoue

Ne pas perdre de message

QUESTION

Avec Spring AMQP et une queue classique, que devient par défaut un message dont le traitement lève une exception dans le listener ?

A. Il est perdu

B. Il est rejoué à l'infini

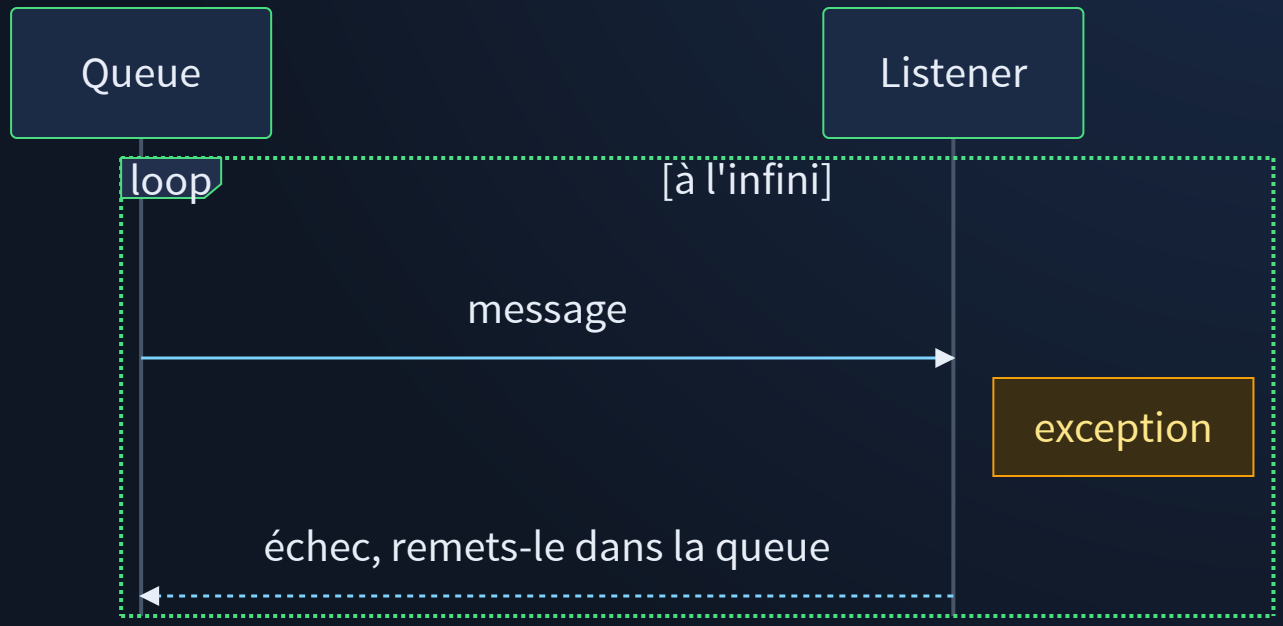
C. Il part dans une dead letter queue

D. Il est renvoyé au producer

Speaker notes

Réponse : B. Laisser répondre, puis dérouler le mécanisme sur la slide suivante.

Ce qui se passe par défaut



Si l'erreur vient d'un bug, le message échouera toujours : il tourne en boucle.

Speaker notes

On parle de poison message : il occupe le listener et remplit les logs. Les autres messages de la queue sont ralentis. Remarque : avec une quorum queue et RabbitMQ 4, le message est abandonné après 20 essais.

Solution, étape 1 : limiter les essais

```
spring.rabbitmq.listener.simple.retry.enabled=true
spring.rabbitmq.listener.simple.retry.max-attempts=3
```

- Spring essaie 3 fois
- Puis il rejette le message pour de bon
- … et le message est **supprimé** !

Speaker notes

Plus de boucle infinie, mais on perd la commande en échec. On veut la garder pour comprendre le problème et la retraiter plus tard : c'est l'étape 2.

Solution, étape 2 : la dead letter queue



Un message rejeté est envoyé à un exchange dédié, qui le range dans une queue à part.

Speaker notes

La dead letter queue est une queue comme les autres : on peut lire les messages, corriger le bug, puis les renvoyer. Il faut surveiller sa taille, sinon personne ne la regarde.

Spring : où envoyer les rejets

```
@Bean
Queue facturationQueue() {
    return QueueBuilder.durable(FACTURATION_QUEUE)
            .deadLetterExchange("fraisdirect.dlx")
            .build();
}
```

Speaker notes

Si la queue facturation existe déjà dans RabbitMQ sans cette option, RabbitMQ refuse de la redéclarer : il faut la supprimer dans l'interface web avant de relancer l'application.

Spring : la queue des rejets

```
@Bean
FanoutExchange deadLetterExchange() {
    return new FanoutExchange("fraisdirect.dlx");
}

@Bean
Queue facturationDlq() {
    return QueueBuilder.durable("facturation.dlq").build();
}
```

Speaker notes

Les mêmes étapes que tout à l'heure : un exchange, puis une queue. Un exchange fanout envoie tout à toutes ses queues, sans regarder la routing key.

Spring : relier les deux

```
@Bean
Binding facturationDlqBinding(
        Queue facturationDlq,
        FanoutExchange deadLetterExchange) {
    return BindingBuilder.bind(facturationDlq)
            .to(deadLetterExchange);
}
```

Pas de `.with(...)` : un exchange fanout ne regarde pas la routing key.

Speaker notes

Comme à l'étape 5 : relier la queue facturation.dlq à l'exchange fraisdirect.dlx.

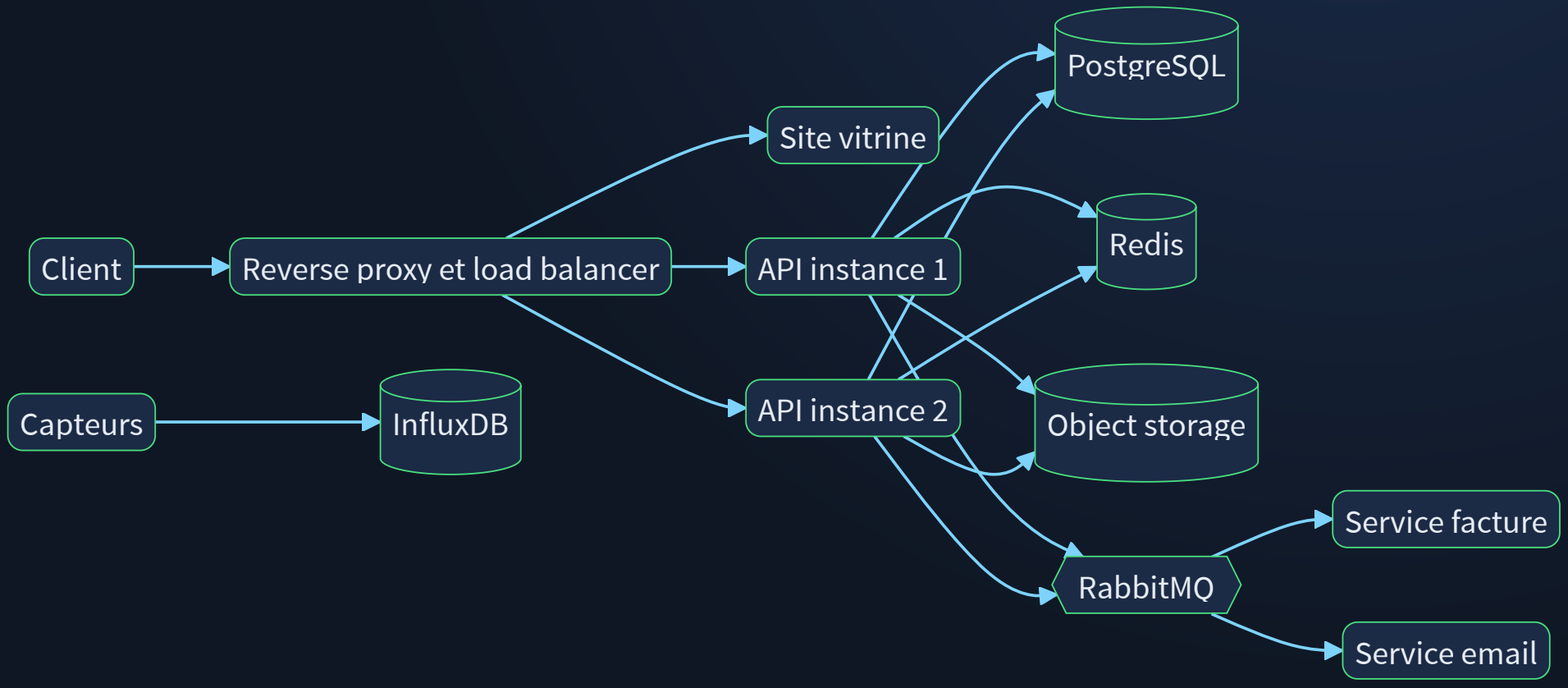
Le parcours complet d'un message en échec

1. Le listener lève une exception
2. Spring réessaie, jusqu'à 3 fois
3. Spring rejette le message
4. RabbitMQ l'envoie à l'exchange `fraisdirect.dlx`
5. Le message attend dans `facturation.dlq`

Speaker notes

Récapitulatif à faire répéter par les étudiants. Il faut les deux étapes : sans limite d'essais, le message n'est jamais rejeté ; sans dead letter queue, il est perdu.

FraisDirect aujourd'hui



Speaker notes

Reprendre le schéma en remontant l'histoire : chaque brique correspond à un problème rencontré pendant le cours. Demander laquelle on supprimerait si le trafic redevenait faible.

À retenir

- Chaque middleware répond à un problème précis
- On l'ajoute quand le problème se pose, pas avant
- Chaque brique en plus doit être installée, sauvegardée et surveillée

Speaker notes

Message principal : avant d'ajouter un middleware, se demander quel problème il résout et qui va l'exploiter.